

Locus: Synthetic Perceptual Modeling

An Offline-First Embedded Knowledge Agent Framework Using Geographic Coordinate Addressing

Toot Toot Engineering
Independent researcher
<https://github.com/antfriend>

2026-05-11 (preprint)

Abstract

We present **Locus**, a framework for *synthetic perceptual modeling* — representing experiential knowledge as a typed graph of perceptual transitions, each anchored to a perceiving agent’s subjective worldview (umwelt). Locus introduces **@LATxLON** geographic coordinate addressing: an authority-free scheme in which every knowledge node is identified by a deterministic latitude/longitude pair requiring no registry, no network connectivity, and no external authority. Knowledge is stored as **toot-bits** organized in **TTDB** (MyMentalPalaceDB), a flat Markdown or LaTeX knowledge graph traversable without a database engine. The framework’s foundational epistemic primitive is the **@PERCEPT paired node**: a before/after transition record that encodes what a perceptual shift was like for a specific agent, not merely that it occurred. Records carry optional **Toot-Bit Epistemic Weight** (TBEW) blocks — confidence, revision count, salience, and recency — from which a derived Epistemic Priority Score (EPS) identifies high-uncertainty, high-salience nodes as primary attention targets, providing a lightweight symbolic approximation of free-energy minimization. The Locus specification suite spans 20 RFCs across four suites: TTDB (knowledge graph format, cursor semantics, typed edges, epistemic weight, experiential perception), TTN (semantic mesh network with trust, compression, and LoRa framing), TTCP (web publishing and interactive 3D globe visualization), and A32 (ESP32 autonomous device framework). Agents on ESP32-class hardware execute deterministic sense-reason-act loops from local TTDB storage — no cloud inference required. We provide a 21-node affective knowledge graph grounded in hippocampal-prefrontal emotional embedding research, and a complete open-source reference implementation. All specifications and firmware are publicly available. —

1 Introduction

Knowledge agents deployed at the edge face a fundamental tension: the richer the knowledge representation, the heavier the runtime dependency. Existing approaches either require cloud inference — introducing latency, connectivity dependency, and data sovereignty concerns — or strip semantics down to lookup tables, sacrificing expressiveness. Locus resolves this tension by separating the *knowledge format* from the *reasoning engine* and grounding both in a novel addressing scheme that requires no central authority.

Locus further addresses a deeper representational gap. Propositional knowledge graphs record *that* a transition occurred but not *what the transition felt like* for a specific perceiving agent. The clinical record knows that a patient experienced symptom relief; the sensor log knows that

temperature crossed a threshold. Neither encodes the agent-bound phenomenological trace — the texture of the perceptual event as it registered in a particular *umwelt*. The **@PERCEPT paired node** ([TTDB-RFC-0006](RFCs/TTDB-RFC-0006-Experiential-Perception-as-Synthetic-Model.md)) fills this gap. Every perceptual event is encoded as a mandatory **@PERCEPT:before** / **@PERCEPT:after** node pair, anchored to the perceiving agent and the transition boundary. The unit of perceptual knowledge is the *edge* between states, not the isolated node. This grounds Locus in biosemiotics (von Uexküll’s *umwelt* concept), active inference (Friston’s free-energy principle), and neurophenomenology (Varela’s embodied cognition), distinguishing it from statistical and latent-space approaches: every inference step is inspectable and auditable from the plain-text TTDB file.

The **@LATxLON** scheme assigns every knowledge record a coordinate-pair identifier derived from its conceptual position on a globe defined per-*umwelt*. Unlike URI schemes — which require an authority registry — or UUID schemes — which are opaque to humans — **@LATxLON** identifiers are deterministic, mnemonic, human-readable, and valid with zero network connectivity. A record at **@LAT30LON-20** is unambiguously located on its *umwelt*’s globe, traversable, linkable, and reproducible from the specification alone. The same identifier assigned to a node today will resolve to the same node in ten years, on any device, with no infrastructure.

TTDB (MyMentalPalaceDB, [TTDB-RFC-0001](RFCs/TTDB-RFC-0001-File-Format.md)) stores knowledge as plain Markdown or LaTeX, parsed with a two-pass indexed reader requiring approximately 12 bytes per record in RAM. **Agent 32 (A32, [A32-RFC-0001](RFCs/A32-RFC-0001-Architecture.md))** runs this parser on ESP32-S3 hardware with 8 MB flash and 2 MB PSRAM, executing a 1-second sense-reason-act cycle entirely from local storage. **TTCP** ([TTCP-RFC-0001](RFCs/TTCP-RFC-0001-Record-Rendering.md) through [TTCP-RFC-0003](RFCs/TTCP-RFC-0003-Link-System-and-Addressability.md)) provides a complementary web-facing layer: an interactive 3D globe renderer that makes the same TTDB knowledge graph explorable by humans via a browser, with a discovery system, guided tour, and **toot:** URI scheme for cross-database linking. **TTN** ([TTN-RFC-0001](RFCs/TTN-RFC-0001.md) through [TTN-RFC-0006](RFCs/TTN-RFC-0006-LoRa-Packet-Framing.md)) provides an optional offline-first LoRa mesh network layer. The Locus framework unifies these four components under a formal 20-RFC specification suite, enabling reproducibility, community extension, and cross-device interoperability without cloud infrastructure.

The remainder of this paper is organized as follows. Section 2 enumerates the key contributions. Section 3 provides a technical overview of the four-tier Locus stack. Section 4 surveys related work. Section 5 presents a conclusion and future directions. Section 6 describes artifact availability.

2 Key Contributions

1. **Synthetic perceptual model (@PERCEPT paired nodes).** [TTDB-RFC-0006](RFCs/TTDB-RFC-0006-Experiential-Perception-as-Synthetic-Model.md) defines the

“ @PERCEPT:before / @PERCEPT:after “ paired node as the fundamental unit of perceptual knowledge. The unit of representation is the *transition* (edge between states), not the isolated node. Agent context is mandatory: every percept is bound to a specific perceiving agent, making the knowledge graph phenomenologically thick and auditable. Distinguishes Locus from both propositional and latent-space approaches: the perceptual trace is assembled from typed primitives, human-readable, and domain-agnostic.

1. **@LATxLON addressing.** An authority-free, deterministic, human-readable namespace for knowledge nodes ([TTDB-RFC-0004](RFCs/TTDB-RFC-0004-Event-ID-and-Collision.md)).

Functions offline with no registration step. Scales from single-device to mesh-networked deployments. The same ID scheme operates across TTDB files, A32 firmware, and TTCP web applications with no translation layer.

2. **TTDB file format.** A flat Markdown/LaTeX knowledge graph specification with formal cursor semantics ([TTDB-RFC-0002](RFCs/TTDB-RFC-0002-Cursor-Semantics.md)), typed directional edges ([TTDB-RFC-0003](RFCs/TTDB-RFC-0003-Typed-Edges.md)), event ID assignment with collision handling ([TTDB-RFC-0004](RFCs/TTDB-RFC-0004-Event-ID-and-Collision.md)), and toot-bit epistemic weight tracking (TBEW, [TTDB-RFC-0005](RFCs/TTDB-RFC-0005-Epistemic-Weight.md)). No database engine required; parseable on a \$5 microcontroller with ~3 KB RAM overhead.
3. **Toot-Bit Epistemic Weight (TBEW).** A per-record metadata block tracking confidence (`conf`, 0–255), revision count (`rev`), salience/query count (`sal`), and last-touch timestamp (`touched`). Derived signal $EPS = sal \times (255 - conf) / 255$ identifies records that are heavily used but poorly understood — the primary attention targets for knowledge refinement. Provides a lightweight symbolic approximation of the free-energy principle using only integer arithmetic, suitable for microcontrollers without floating-point units.
4. **Umwelt formalism.** Operationalization of von Uexküll’s biosemiotic umwelt concept as a firmware-level device identity ([A32-RFC-0001](RFCs/A32-RFC-0001-Architecture.md)). Two physically identical ESP32 devices loaded with different TTDB files exhibit different behaviors, different reasoning paths, and different knowledge perspectives — they inhabit different umwelten. The umwelt is not a runtime property but a design-time binding: it is authored into the TTDB file and instantiated at firmware boot.
5. **TTCP web publishing suite.** Three RFCs specify how TTDB knowledge graphs are rendered and explored in a browser: a record rendering pipeline supporting Markdown, LaTeX, and KaTeX with epistemic weight display and media handling ([TTCP-RFC-0001](RFCs/TTCP-RFC-0001-Record-Rendering.md)), an interactive 3D knowledge globe with discovery system, guided tour, scene playback, and multi-database navigation ([TTCP-RFC-0002](RFCs/TTCP-RFC-0002-Globe-and-Navigation.md)), and a `toot`: URI scheme for cross-database deep linking ([TTCP-RFC-0003](RFCs/TTCP-RFC-0003-Link-System-and-Addressability.md)). The same TTDB file that runs on an ESP32 is explorable as a static web application with no server and no conversion step.
6. **20-RFC specification suite.** Formal specifications across four suites: TTN×6 (semantic mesh network with typed edges, trust/reputation, semantic compression, and LoRa framing), TTDB×6 (file format, cursor, typed edges, event IDs, epistemic weight, experiential perception), TTCP×3 (record rendering, globe visualization, link system), and A32×4 + Amendment-A (ESP32 firmware architecture, TTDB streaming parser, agent loop, Claude Code project setup, TBEW C++ parser extension). Full index: RFCs/INDEX.md. Enables reproducibility and community fork/extend workflows.
7. **Affective knowledge graph.** A 21-node TTDB instance ([feelings_ttdb.md](https://antfriend.github.io/?ttdb=feelings_ttdb.md)) mapping feelings, emotions, dispositions, and intents on a valence/arousal coordinate space. Grounded in Ma & Kragel (2026) map-like representations of emotion knowledge in hippocampal-prefrontal systems (*Nature Communications*, 17, 1518). The umwelt node at [origin @LAT0LON0](https://antfriend.github.io/?ttdb=feelings_ttdb.md&toot=lat0lon0) represents the experiencing subject; states extend outward

by valence (north = positive, south = negative) and object of affect (east = other-directed, west = self-directed). Deployable to ESP32 as a standalone affective reasoning module.

3 System Overview

The Locus stack has four tiers. Figure descriptions are provided inline; rendered globe views are available at <https://antfriend.github.io/>.

Storage tier (TTDB): A flat text file (Markdown or LaTeX) with three structural zones: a mmpdb YAML block (database properties including `db_id`, `db_name`, `coord_increment`, `umwelt` definition, `typed_edges` configuration, and optional librarian settings); a cursor YAML block (selection state, preview, agent note, Graphviz dot fragment, and librarian query history); and a sequence of record sections separated by `--` delimiters. Each record opens with a header line of the form “`@LAT10LON-10 | created:1775260800 | updated:1775260800 | relates:feels>@LAT0LON0,resonates_wit`” followed by an optional [ew] epistemic weight block, and then a free-text Markdown or LaTeX body. Perceptual knowledge is encoded as `@PERCEPT:before / @PERCEPT:after` paired nodes — the before/after pair is mandatory and must be agent-bound; orphaned percept nodes are specification errors ([TTDB-RFC-0006](RFCs/TTDB-RFC-0006-Experiential-Perception-as-Synthetic-Model.md)).

Coordinate IDs are assigned deterministically: a record conceptually positioned at valence +10, arousal -10 (mild positive, self-oriented) in the affective umwelt receives ID `@LAT10LON-10`. Collision is resolved by a configurable policy (e.g., `southeast_step`: increment both axes by one step and retry). Once assigned, IDs are permanent; superseded records link to successors via a `revises>` typed edge ([TTDB-RFC-0004](RFCs/TTDB-RFC-0004-Event-ID-and-Collision.md)). The affective knowledge graph ([`feelings_ttdb.md`](https://antfriend.github.io/?ttdb=feelings_ttdb.md)) uses `coord_increment`: {`lat`: 10, `lon`: 10} with 21 nodes spanning `@LAT-50LON-40` (deep grief) to `@LAT40LON30` (elation), with the umwelt at `@LAT0LON0`.

Agent tier (A32): Firmware for ESP32-class hardware ([A32-RFC-0001](RFCs/A32-RFC-0001-Architecture.md) through [A32-RFC-0004](RFCs/A32-RFC-0004-Claude-Code-Setup.md)). At boot, a two-pass streaming parser ([A32-RFC-0002](RFCs/A32-RFC-0002-TTDB-Storage.md)) builds an in-memory index: Pass 1 scans the file recording each record’s coordinate ID and byte offset (12 bytes per index entry, stored in PSRAM); Pass 2 reads record bodies on demand by seeking to the stored offset. Memory budget: approximately 3 KB fixed overhead plus 12 bytes per record; a 1,000-node TTDB occupies roughly 15 KB of PSRAM index. The agent loop ([A32-RFC-0003](RFCs/A32-RFC-0003-Agent-Loop.md)) executes `sense → reason → act → wait` at a configurable interval (default 1 second). Sense reads registered sensors and quantizes their values to TTDB coordinates using the `coord_increment` grid. Reason finds the nearest record in the index, reads its typed edges, and selects action edges (`triggers`, `navigates_to`, `inhibits`, `requires`, `logs`). Act dispatches each action through the registered actuator. Observation logs are written append-only to a separate CSV file; the TTDB file itself is never modified at runtime.

Network tier (TTN): An optional LoRa mesh network layer for multi-device deployments ([TTN-RFC-0001](RFCs/TTN-RFC-0001.md) through [TTN-RFC-0006](RFCs/TTN-RFC-0006-LoRa-Packet-Framing.md)). Nodes communicate via semantic events rather than raw messages, following five core principles: meaning over messages, offline-first and partition-tolerant, local data sovereignty, transport agnosticism, and explicit AI invocation only. The minimal LoRa packet frame ([TTN-RFC-0006](RFCs/TTN-RFC-0006-LoRa-Packet-Framing.md)) is 253 bytes maximum: “`[SOF=0xA5] [VER=0x01] [FLAGS] [SRC_ID:2] [DST_ID:2] [TYPE] [SEQ] [LEN] [PAYLOAD:0-240] [CRC16:2] [EOF=0x01]`” Ultra-low-bandwidth semantic compression ([TTN-RFC-0004](RFCs/TTN-RFC-0004-Semantic-Compression.md)).

represents common events as single-byte tokens: P (presence), S? (status request), OK, ERR, SOS, T:x (temperature), H:x (humidity), B:x' (battery); gateways expand tokens to full semantic events. Trust is modeled locally per device via signed edge corroboration ([TTN-RFC-0005](RFCs/TTN-RFC-0005-Trust-and-Reputation.md)); no global ledger or central trust authority is required.

Publishing tier (TTCP): A browser-based layer that renders any TTDB file as an interactive 3D knowledge globe at <https://antfriend.github.io/>. The three-zone parse model ([TTCP-RFC-0001](RFCs/TTCP-RFC-0001-Record-Rendering.md)) ingests `mmpdb`, `cursor`, and record zones, producing per-record HTML with Markdown or KaTeX rendering, epistemic weight display, and media handling (video, iframe, images). The globe ([TTCP-RFC-0002](RFCs/TTCP-RFC-0002-Globe-and-Navigation.md)) projects each record to its @LATxLONy sphere coordinates; node color is deterministic per record ID (hash-based HSL); edges are arcs connecting related nodes; a discovery system tracks visited records in local storage; a guided tour auto-advances on a configurable timer; scene records define playback sequences with transition animations. The `toot:` URI scheme ([TTCP-RFC-0003](RFCs/TTCP-RFC-0003-Link-System-and-Addressability.md)) provides cross-database deep linking: `toot://db_alias/lat10lon-10` addresses a specific record in a named database; URL parameters `?ttdb=feelings_ttdb.md&toot=lat10lon-10` encode the same selection, enabling bookmarkable, shareable links to individual knowledge nodes. No server is required; TTCP implementations run as static web applications reading TTDB files directly from the same origin.

4 Related Work

Knowledge graphs on embedded hardware. Symbolic knowledge representation on microcontroller-class hardware has received limited attention. Prior approaches to RDF on constrained devices (e.g., μ -RDF-Store on sensor nodes) require a SPARQL query evaluator whose memory footprint substantially exceeds the ESP32's available RAM. Protobuf-based schemas used in micro-ROS enable structured messages but do not constitute a semantic knowledge graph with typed edges and coordinate addressing. TTDB achieves expressiveness comparable to RDF through a 26-type edge taxonomy ([TTN-RFC-0002](RFCs/TTN-RFC-0002-Typed-Edges.md)) while requiring only a 3 KB streaming parser that completes indexing in a single file pass. The flat Markdown format means a knowledge graph can be authored in any text editor, inspected without tooling, and version-controlled as plain text.

Authority-free naming. The challenge of naming knowledge nodes without a central registry has been addressed through content addressing (IPFS/IPLD content identifiers — CIDs derived from hash of content), decentralized identifiers (W3C DIDs, which require a resolver and method-specific verification infrastructure), and persistent URLs (PURLs, which require an HTTP redirect service). Content addresses are opaque hash strings that cannot be predicted before content is written; DIDs require resolver infrastructure; PURLs require HTTP. @LATxLON identifiers are derived deterministically from conceptual position in a per-umwelt coordinate space — a node at @LAT30LON20 can be assigned before its content is written, with zero infrastructure, and the same identifier resolves identically on any future device from the specification alone. The closest prior work is the W3C Semantic Sensor Network (SSN) ontology's use of WGS84 coordinates for physical sensor locations; @LATxLON differs in operating on conceptual rather than physical space, and in binding the coordinate frame to the perceiving agent's umwelt rather than a shared geographic reference.

Umwelt in robotics and AI. Uexküll's umwelt concept — the subjective perceptual world each organism constructs from its sensory apparatus — has been applied metaphorically in robotics

(Pfeifer and Scheier’s embodied cognition, Rolf et al. on sensor-motor contingencies) and in philosophy of mind (Metzinger’s phenomenal self-model). In robotics, "umwelt" typically refers informally to the sensor-actuator coupling of a specific embodiment without a formal specification or knowledge binding. Locus operationalizes umwelt as a firmware-level design-time binding: the `umwelt` block in the TTDB `mmpdb` header specifies `umwelt_id`, `role`, `perspective`, `scope`, and `constraints` — a complete formal identity specification. Two physically identical ESP32 devices loaded with different TTDB files inhabit different umwelten, exhibiting different behaviors and knowledge perspectives. This constitutes, to our knowledge, the first formalization of umwelt as a device identity specification in an embedded systems framework.

Offline AI agents and TinyML. TinyML frameworks (TensorFlow Lite Micro, Edge Impulse) enable neural inference on microcontrollers but require offline training, fixed-schema model weights, and floating-point computation. Knowledge distillation and INT8 quantization reduce model size but preserve the fundamental opacity of neural representations: the agent cannot explain *why* it selected an action. Rule-based systems (CLIPS, Drools) achieve inspectability but require a runtime engine exceeding ESP32 RAM. Locus agents use deterministic symbolic graph traversal: no training data, no model weights, no floating-point, fully inspectable at every step. The knowledge graph itself — in plain Markdown — is the complete specification of agent behavior: a human reading the TTDB file can trace every possible agent action without executing any code.

Epistemic agents and the free-energy principle. Friston’s free-energy principle formalizes perception and action as inference: agents minimize surprise by updating beliefs (perception) and sampling preferred observations (action). Expected free-energy minimization gives rise to epistemic foraging — agents preferentially exploring high-uncertainty, high-salience regions of belief space (Schwartenbeck et al. 2019). TBEW recovers this intuition in symbolic form: the Epistemic Priority Score $EPS = \text{sal} \times (255 - \text{conf}) / 255$ identifies nodes that are frequently queried (high salience) but poorly settled (low confidence), directing agent attention without matrix operations or probability distributions. All arithmetic is unsigned integer, executable on a bare-metal microcontroller without a floating-point unit, making TBEW the first lightweight symbolic approximation of epistemic active inference suitable for ESP32-class hardware.

5 Conclusion

Locus establishes a novel class of embedded knowledge agent: fully offline, deterministic, inspectable, and grounded in a formal biosemiotic theory of perception. The @PERCEPT paired node addresses a representational gap that propositional knowledge graphs cannot fill — encoding not merely that a perceptual transition occurred but what it was like for a specific perceiving agent. The @LATxLON addressing scheme eliminates the authority bottleneck without sacrificing human readability or offline operation. The 20-RFC specification suite ensures that every design decision is documented, reproducible, and community-extendable. The TTCP publishing tier closes the loop between embedded agent reasoning and human knowledge exploration: the same TTDB file that drives a sensor-actuator loop on a \$5 microcontroller is viewable as an interactive 3D knowledge globe at <https://antfriend.github.io/>.

Future work includes formal evaluation of agent reasoning quality on benchmark knowledge graphs, multi-device TTN mesh deployments with trust gradient experiments, and clinical informatics applications using UMLS CUI grounding in the medical knowledge graph. The @PERCEPT formalism and outcome node schema (Actions 3–4 in locus_immediate_next_actions.md) require resolution before Paper D (AMIA) submission. A JOSS software paper is planned following Zenodo deposit.

6 Availability

All specifications, reference firmware, and TTDB collections are available under open licenses.

- **RFC suite (20 RFCs):** RFCs/INDEX.md — full index with status, dependencies, and cross-references. GitHub: <https://github.com/antfriend/antfriend.github.io/tree/master/RFCs>
- **TTCP web viewer and TTDB collections:** <https://antfriend.github.io/> — live viewer, interactive 3D globe, all example databases
- **Affective knowledge graph (21 nodes):** https://antfriend.github.io/?ttdb=feelings_ttdb.md — explore the valence/arousal coordinate space; begin at the [umwelt origin](https://antfriend.github.io/?ttdb=feelings_ttdb.md&toot=lat0lon0) or the [Joy node](https://antfriend.github.io/?ttdb=feelings_ttdb.md&toot=lat30lon30)
- **A32 mega TTDB (reference knowledge base):** https://antfriend.github.io/?ttdb=a32_mega_ttdb.md
- **GitHub repository:** <https://github.com/antfriend/antfriend.github.io> — source for all TTDB files, TTCP viewer, Python utilities, and build tooling
- **Zenodo dataset deposit (DOI):** [to be added after deposit — see Action 2 in locus_immediate_next_actions.md]

References

- [1] Uexküll, J. von (1909). *Umwelt und Innenwelt der Tiere*. Gustav Fischer.
- [2] Ma, Y. & Kragel, P.A. (2026). Map-like representations of emotion knowledge in hippocampal-prefrontal systems. *Nature Communications*, 17, 1518. <https://doi.org/10.1038/s41467-025-68240-z>
- [3] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.
- [4] Varela, F.J., Thompson, E., & Rosch, E. (1991). *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.
- [5] Peirce, C.S. (1903). Pragmatism as the logic of abduction. Lecture VII, *Harvard Lectures on Pragmatism*. In *Collected Papers*, vol. 5.
- [6] Schwartenbeck, P., Passecker, J., Hauser, T.U., FitzGerald, T.H., Kronbichler, M., & Friston, K.J. (2019). Computational mechanisms of curiosity and goal-directed exploration. *eLife*, 8, e41703. <https://doi.org/10.7554/eLife.41703>
- [7] David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., Warden, P., & Rhodes, R. (2021). TensorFlow Lite Micro: Embedded machine learning for TinyML systems. *Proceedings of Machine Learning and Systems*, 3, 800–811.
- [8] Bodenreider, O. (2004). The Unified Medical Language System (UMLS): integrating biomedical terminology. *Nucleic Acids Research*, 32(suppl_1), D267–D270.